

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Parte 2 – Prof. Daniele Radicioni

Lab 4

Modellazione del Linguaggio con N-grammi
Confronto tra linguaggio dei social media e linguaggio letterario

Gruppo di lavoro:

Studente	Matricola
Alessandro Olivero	915069
Samuele Iudice	1235245
Andrea Franco	1226739

Docente: Prof. Daniele Radicioni

Anno Accademico: 2025/2026

Indice

1	Introduzione	2
2	Background Teorico	2
2.1	N-grammi	2
2.2	Modello MLE (Maximum Likelihood Estimation)	2
2.3	Smoothing di Laplace (Add-1)	3
2.4	Perplexity	3
2.5	Log-likelihood	3
2.6	Type-Token Ratio (TTR)	3
3	Dataset e Pre-elaborazione	4
3.1	Dataset	4
3.2	Pipeline di pulizia	4
4	Analisi Statistica dei Corpus	4
4.1	Tokenizzazione e vocabolario	4
4.2	Rimozione delle stopwords e analisi dei bigrammi	5
5	Addestramento dei Modelli Linguistici	5
5.1	Preparazione e fitting	5
5.2	Probabilità condizionali	6
6	Generazione di Testo	7
7	Perplexity Cross-Domain	7
7.1	Configurazione dell'esperimento	7
7.2	Limiti della perplexity come classificatore	8
8	Classificatore con Log-Likelihood	9
9	MLE vs. Laplace: confronto diretto	10
10	Confronto $n = 1, 2, 3$	10
11	Analisi Linguistica Comparata	11
11.1	Type-Token Ratio	11
11.2	Lunghezza media delle frasi	12
12	Risultati e Discussione	12
12.1	Perché entrambi i classificatori raggiungono il 100%	13
12.2	Il ruolo dello smoothing: non solo evitare probabilità zero	13
12.3	L'ordine n non aiuta linearmente: il costo della sparsity	14
12.4	Sintesi tabellare	14
12.5	Osservazioni principali	14
13	Conclusioni	15
14	Dichiarazione sui Contributi di IA	15

1 Introduzione

Questo laboratorio ha l'obiettivo di esplorare la **modellazione del linguaggio** tramite N-grammi, confrontando due domini linguistici molto diversi tra loro:

- **Linguaggio Twitter**: messaggi brevi, informali, ricchi di slang, abbreviazioni ed emoji.
- **Linguaggio letterario**: testo formale tratto dal romanzo *Emma* di Jane Austen (1816), caratterizzato da frasi lunghe e sintassi elaborata.

L'analisi copre i seguenti aspetti:

1. Statistiche descrittive sui corpus (vocabolario, TTR, lunghezza frasi).
2. Estrazione e analisi di bigrammi e trigrammi.
3. Addestramento di modelli linguistici MLE e Laplace.
4. Generazione automatica di testo.
5. Classificazione di frasi tramite perplexity e log-likelihood.

L'intero flusso sperimentale procede in tre fasi: i due corpus (Twitter e letterario) vengono puliti e tokenizzati separatamente, mantenendo ciascun tweet/frase come sequenza a sé (Sezione 5 spiega perché questo passaggio è cruciale); le sequenze vengono usate per addestrare due coppie di modelli n-gram (MLE e Laplace) indipendenti; dai modelli addestrati si diramano infine i tre utilizzi a valle discussi nel resto della relazione: generazione di testo, classificazione di dominio tramite perplexity e classificazione tramite log-likelihood.

2 Background Teorico

2.1 N-grammi

Un **N-gramma** è una sequenza contigua di N token (parole) estratta da un testo. I casi più comuni sono:

- **Unigramma** ($N = 1$): singola parola.
- **Bigramma** ($N = 2$): coppia di parole consecutive.
- **Trigramma** ($N = 3$): tripletta di parole consecutive.

Gli N-grammi sono la base dei **modelli linguistici statistici**: stimano la probabilità di una parola dato il contesto delle $N-1$ parole precedenti.

2.2 Modello MLE (Maximum Likelihood Estimation)

Il modello MLE stima la probabilità condizionale con la frequenza relativa osservata nel corpus:

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

dove $C(\cdot)$ indica il conteggio nel corpus. Il problema principale del MLE è che assegna probabilità **zero** a qualsiasi sequenza non vista durante il training: basta una parola fuori dal vocabolario per rendere l'intera frase impossibile secondo il modello.

2.3 Smoothing di Laplace (Add-1)

Per ovviare al problema dello zero, lo **smoothing di Laplace** aggiunge 1 a ogni conteggio:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

dove $|V|$ è la dimensione del vocabolario. In questo modo nessuna sequenza ha probabilità nulla, al costo di ridistribuire parte della probabilità verso eventi non osservati.

2.4 Perplexity

La **perplexity** misura quanto un modello è “sorpreso” da un testo di test. Un modello buono assegna alta probabilità al testo, producendo bassa perplexity.

$$\text{PP}(W) = P(w_1 w_2 \cdots w_N)^{-\frac{1}{N}} = \left(\prod_{i=1}^N \frac{1}{P(w_i \mid w_{i-1})} \right)^{\frac{1}{N}}$$

Più bassa è la perplexity, migliore è il modello per quel testo.

2.5 Log-likelihood

La **log-likelihood** di una sequenza misura quanto il modello “conosce” quel testo. Si calcola come somma dei logaritmi delle probabilità dei singoli token:

$$\ell(W) = \sum_{i=1}^N \log P(w_i \mid w_{i-1})$$

Il valore è sempre ≤ 0 : più è vicino a zero (meno negativo), meglio il modello descrive il testo. Rispetto alla perplexity è numericamente più stabile ed è diretta da usare come classificatore: si assegna un testo al dominio che gli assegna log-likelihood più alta.

2.6 Type-Token Ratio (TTR)

Il **TTR** misura la ricchezza lessicale di un testo:

$$\text{TTR} = \frac{|\text{Types}|}{|\text{Tokens}|}$$

dove *Types* sono le parole uniche e *Tokens* sono tutte le occorrenze. Valori alti indicano vocabolario vario; valori bassi indicano ripetitività.

3 Dataset e Pre-elaborazione

3.1 Dataset

- **Twitter:** corpus `twitter_samples` di NLTK, composto da 5000 tweet positivi e 5000 tweet negativi (10000 tweet in totale).
- **Letterario:** testo integrale di *Emma* di Jane Austen (1816), disponibile nel corpus `gutenberg` di NLTK.

3.2 Pipeline di pulizia

I due corpus richiedono trattamenti diversi a causa della natura del linguaggio.

Pulizia dei tweet e del testo letterario

```
1 def pulisci_tweet(testo):
2     testo = re.sub(r'http\S+', '', testo)           # rimuove URL
3     testo = re.sub(r'@\w+', '', testo)             # rimuove menzioni
4     testo = re.sub(r'#', '', testo)                # rimuove simbolo hashtag
5     testo = re.sub(r'[a-zA-Z\s]', '', testo)       # solo lettere
6     testo = testo.lower().strip()
7     return testo
8
9 def pulisci_letterario(testo):
10    testo = re.sub(r'[a-zA-Z\s]', '', testo)
11    testo = testo.lower().strip()
12    return testo
```

Per i tweet si rimuovono URL, menzioni e hashtag prima di normalizzare il testo. Per il testo letterario si rimuovono solo i caratteri non alfabetici, mantenendo la struttura originale.

4 Analisi Statistica dei Corpus

4.1 Tokenizzazione e vocabolario

Tokenizzazione e calcolo del vocabolario

```
1 tokens_tweet = word_tokenize(testo_tweet_unito)
2 tokens_lett  = word_tokenize(testo_lett_pulito)
3
4 vocab_tweet_set = set(tokens_tweet)
5 vocab_lett_set  = set(tokens_lett)
```

Tabella 1: Statistiche descrittive dei corpus

Statistica	Twitter	Letterario
Token totali	94 706	158 267
Parole uniche (types)	12 024	9 305
TTR grezzo*	0.1270	0.0588
Lunghezza media frase	16.0 token	25.6 token

*Calcolato sull'intero corpus di ciascun dominio: non è un confronto valido, perché il TTR scende all'aumentare dei token. Il confronto corretto, su campioni di uguale dimensione, è in Sezione 11.1.

Le parole uniche riflettono la dimensione del vocabolario di ciascun corpus così com'è, mentre il testo letterario ha frasi molto più lunghe in media. Il TTR grezzo qui sopra non è direttamente confrontabile tra i due domini (Sezione 11.1 presenta il confronto corretto).

4.2 Rimozione delle stopwords e analisi dei bigrammi

Rimozione stopwords e calcolo bigrammi/trigrammi

```

1 stop = set(stopwords.words('english'))
2
3 tokens_tweet_filtrati = [t for t in tokens_tweet if t not in stop]
4 tokens_lett_filtrati = [t for t in tokens_lett if t not in stop]
5
6 bi_tweet_filtrati = Counter(ngrams(tokens_tweet_filtrati, 2))
7 bi_lett_filtrati = Counter(ngrams(tokens_lett_filtrati, 2))

```

Rimuovendo le stopwords emergono i bigrammi semanticamente significativi di ciascun dominio:

- **Twitter:** bigrammi legati a emozioni (*feel good, love much*), auguri (*happy birthday*) e relazioni (*miss much*).
- **Letterario:** bigrammi tipici del romanzo vittoriano (*mr knightley, miss woodhouse, emma thought*).

5 Addestramento dei Modelli Linguistici

5.1 Preparazione e fitting

NLTK fornisce `padded_everygram_pipeline`, che aggiunge automaticamente i token di inizio (<s>) e fine (</s>) frase e genera tutti gli N-grammi fino a n . Ogni tweet e ogni frase del romanzo viene passato come sequenza a sé stante, in modo che il padding (e quindi i bigrammi generati) non attraversi il confine tra un tweet/frase e il successivo:

Addestramento modelli MLE e Laplace per bigrammi ($n = 2$)

```
1 n = 2
2
3 sequenze_tweet = [word_tokenize(t) for t in tweets_puliti]
4 frasi_lett_raw = sent_tokenize(testo_letterario_raw)
5 sequenze_lett = [word_tokenize(pulisci_letterario(f)) for f in frasi_lett_raw]
6 sequenze_lett = [s for s in sequenze_lett if s]
7
8 # Modelli MLE: per generazione e probabilita' condizionali
9 train_tw_mle, vocab_tw_mle = padded_everygram_pipeline(n, sequenze_tweet)
10 modello_tweet = MLE(n)
11 modello_tweet.fit(train_tw_mle, vocab_tw_mle)
12
13 train_lt_mle, vocab_lt_mle = padded_everygram_pipeline(n, sequenze_lett)
14 modello_lett = MLE(n)
15 modello_lett.fit(train_lt_mle, vocab_lt_mle)
16
17 # Modelli Laplace: per perplexity (gestisce parole mai viste)
18 train_tw_lp, vocab_tw_lp = padded_everygram_pipeline(n, sequenze_tweet)
19 modello_tweet_lp = Laplace(n)
20 modello_tweet_lp.fit(train_tw_lp, vocab_tw_lp)
21
22 train_lt_lp, vocab_lt_lp = padded_everygram_pipeline(n, sequenze_lett)
23 modello_lett_lp = Laplace(n)
24 modello_lett_lp.fit(train_lt_lp, vocab_lt_lp)
```

Si addestrano due varianti per ciascun corpus:

- **MLE**: usato per la generazione di testo e il calcolo delle probabilità condizionali.
- **Laplace**: usato per la perplexity, poiché MLE assegna probabilità 0 a parole mai viste, rendendo la perplexity infinita.

Nota su una correzione di correttezza: in una prima versione, tutti i tweet (o tutte le frasi del romanzo) venivano concatenati in un'unica sequenza prima di essere passati a `padded_everygram_pipeline`. Questo produceva un solo `<s>/</s>` per l'intero corpus, e generava bigrammi spuri a cavallo tra la fine di un tweet e l'inizio del successivo (una relazione che nella realtà non esiste). Passare una sequenza per ogni tweet/frase risolve il problema.

5.2 Probabilità condizionali

Calcolo probabilità condizionali con MLE

```
1 # Modello Twitter
2 print(modello_tweet.score('love', ['i'])) # P('love' | 'i')
3 print(modello_tweet.score('happy', ['so'])) # P('happy' | 'so')
4
5 # Modello Letterario
6 print(modello_lett.score('mr', ['said'])) # P('mr' | 'said')
7 print(modello_lett.score('been', ['had'])) # P('been' | 'had')
```

Le probabilità condizionali riflettono il registro linguistico: il modello Twitter assegna alta probabilità a *love* dopo *i*, mentre il letterario assegna alta probabilità a *mr* dopo *said* e a *been* dopo *had*, strutture tipiche della prosa ottocentesca.

6 Generazione di Testo

Generazione di testo con filtraggio degli artefatti

```
1 def genera_testo(modello, n_parole=30):
2     testo_raw = modello.generate(n_parole, random_seed=42)
3
4     testo_filtrato = []
5     for token in testo_raw:
6         # scarta token speciali del padding
7         if token in ['<s>', '</s>', '<UNK>']:
8             continue
9         # scarta lettere singole (artefatti), tranne 'i' e 'a'
10        if len(token) == 1 and token not in ['i', 'a']:
11            continue
12        testo_filtrato.append(token)
13
14    return ' '.join(testo_filtrato)
15
16 # Esempi di output
17 print(genera_testo(modello_tweet, 40)) # stile Twitter
18 print(genera_testo(modello_lett, 40))  # stile letterario
```

Il metodo `generate()` campiona parola per parola usando le probabilità del bigramma. I token `<s>`, `</s>` e `<UNK>` sono artefatti del padding e vengono rimossi. Le lettere singole (diverse da *i* e *a*) sono artefatti di tokenizzazione e vengono anch'esse filtrate.

Esempi di testo generato (seed=42, 40 parole):

- *Stile Twitter*: frasi brevi, emotivamente cariche, con ripetizioni tipiche dei social.
- *Stile letterario*: frasi più lunghe, con lessico formale e costruzioni sintattiche elaborate tipiche di Austen.

7 Perplexity Cross-Domain

7.1 Configurazione dell'esperimento

L'idea è usare la perplexity come classificatore: dato un testo, il modello che gli assegna **perplexity più bassa** è quello del dominio di appartenenza.

Si usano 8 frasi di test (4 da Twitter, 4 letterarie) e si calcola la perplexity con entrambi i modelli Laplace. `perplexity()` di NLTK si aspetta in input una sequenza di n-grammi (non i token grezzi), quindi le frasi vengono prima riportate in bigrammi con padding tramite `pad_both_ends`. Poiché i vocabolari hanno dimensioni molto diverse, il verdetto si basa sulla perplexity normalizzata per la dimensione del vocabolario:

Classificazione tramite perplexity normalizzata

```
1 def calcola_perplexity(modello, tokens, n=2):
2     ngr = list(ngrams(pad_both_ends(tokens, n=n), n))
3     return modello.perplexity(ngr)
4
5 for nome, frase in frasi_test.items():
6     tokens_frase = word_tokenize(frase)
7     pp_tw = calcola_perplexity(modello_tweet_lp, tokens_frase, n)
8     pp_lett = calcola_perplexity(modello_lett_lp, tokens_frase, n)
9
10    # Normalizza per la dimensione del vocabolario
11    pp_tw_n = pp_tw / len(modello_tweet_lp.vocab)
12    pp_lett_n = pp_lett / len(modello_lett_lp.vocab)
13
14    verdetto_atteso = "TW" if nome.startswith("tw") else "LT"
15    verdetto = "TW" if pp_tw_n < pp_lett_n else "LT"
16    corretto = "corretto" if verdetto == verdetto_atteso else "errato"
```

Tabella 2: Perplexity Laplace (non normalizzata) sulle 8 frasi di test

Frase	PP Twitter	PP Letterario	Verdetto
tw_pos_1	128.1	538.7	TW ✓
tw_pos_2	1425.3	3434.1	TW ✓
tw_neg_1	235.7	2151.0	TW ✓
tw_neg_2	577.8	1375.3	TW ✓
lett_1	6321.1	290.1	LT ✓
lett_2	797.7	159.3	LT ✓
lett_3	3821.2	442.0	LT ✓
lett_4	2605.8	126.2	LT ✓

Con la perplexity calcolata correttamente sui bigrammi, il verdetto (basato sui valori normalizzati) è corretto per tutte le 8 frasi: **accuratezza** $8/8 = 100\%$.

7.2 Limiti della perplexity come classificatore

Su questo specifico set di frasi la perplexity normalizzata raggiunge il 100% di accuratezza, tanto quanto la log-likelihood (Sezione 8): i due domini sono linguisticamente così distanti che entrambi i metodi separano correttamente ogni frase di test. Il vantaggio della log-likelihood non è quindi visibile in questo esperimento, ma resta valido sul piano teorico: la normalizzazione per $|V|$ usata per la perplexity è una correzione empirica, non una pratica standard, mentre la log-likelihood confronta direttamente due modelli sullo stesso testo senza bisogno di questo aggiustamento ad hoc.

8 Classificatore con Log-Likelihood

Calcolo della log-likelihood di una sequenza

```
1 def log_likelihood(modello, tokens):
2     # Log-prob. media per token. Richiede Laplace: con MLE un bigramma
3     # mai visto avrebbe prob. 0 e il floor sotto dominerebbe la somma.
4     score = 0.0
5     for i in range(1, len(tokens)):
6         contesto = [tokens[i-1]]
7         parola = tokens[i]
8         prob = modello.score(parola, contesto)
9         if prob > 0:
10            score += math.log(prob)
11        else:
12            score += math.log(1e-10) # floor, quasi mai attivo
13    return score / (len(tokens) - 1)
14
15 def classifica_loglik(testo, mod_tw, mod_lt):
16     tokens = word_tokenize(testo.lower())
17     ll_tw = log_likelihood(mod_tw, tokens)
18     ll_lett = log_likelihood(mod_lt, tokens)
19     # vince il modello con log-likelihood piu' alta
20     verdetto = "TWITTER" if ll_tw > ll_lett else "LETTERARIO"
21     return verdetto, ll_tw, ll_lett
22
23 for testo in testi_classificare:
24     classifica_loglik(testo, modello_tweet_lp, modello_lett_lp) # Laplace, non
    MLE
```

Nota su una correzione di correttezza: in una prima versione `classifica_loglik` veniva chiamato con i modelli MLE (`modello_tweet`, `modello_lett`), in contraddizione con la docstring della funzione, che assume uno smoothing di tipo Laplace. Con MLE, qualunque bigramma mai visto in training ha probabilità esattamente 0, il floor manuale ($\log 10^{-10} \approx -23$) scatta e finisce per dominare l'intera somma: il classificatore diventava di fatto “chi ha meno bigrammi mai visti vince” invece di un confronto genuino di verosimiglianza. Usare i modelli Laplace (che non assegnano mai probabilità esattamente nulla) rende il floor un caso limite che in pratica non si attiva più. Si è inoltre normalizzato il punteggio dividendo per il numero di transizioni, per ottenere una log-likelihood media per token confrontabile anche tra frasi di lunghezza diversa.

Il classificatore assegna il testo al modello con log-likelihood più alta (meno negativa). Rispetto alla perplexity normalizzata, questo approccio ha alcuni vantaggi metodologici:

- Non richiede di dividere per la dimensione del vocabolario per rendere i due modelli confrontabili.
- Un floor di 10^{-10} (invece di $P = 0$) evita i valori $-\infty$ sui bigrammi mai visti, pur restando un caso limite che con Laplace non si attiva quasi mai.

Tabella 3: Log-likelihood media per token (modelli Laplace) sulle 8 frasi di test

Frase	LL Twitter	LL Letterario	Verdetto
i love you so much thank you	-5.289	-7.138	TWITTER
she had been very well disposed towards him	-9.205	-6.143	LETTERARIO
i miss you so much cant stop thinking	-6.476	-8.280	TWITTER
it was a very good thing to have done	-6.828	-5.110	LETTERARIO
happy birthday love you so much	-5.622	-7.727	TWITTER
she was not a woman of many words	-8.288	-6.323	LETTERARIO
i cant stop smiling so happy	-7.662	-8.890	TWITTER
mr knightley had always been very fond of her	-9.062	-5.882	LETTERARIO

Tutti gli 8 verdeti coincidono con il dominio atteso: **accuratezza** $8/8 = 100\%$, in linea con quanto ottenuto dalla perplexity normalizzata (Sezione 7). Si noti che questo set di frasi è diverso (per quanto in parte sovrapposto) da quello usato in Sezione 7 per la perplexity.

9 MLE vs. Laplace: confronto diretto

Confronto MLE e Laplace su parola mai vista

```

1 parola_rara = "xyzword"
2 print(modello_tweet.score(parola_rara, ['i']))      # MLE -> 0.0
3 print(modello_tweet_lp.score(parola_rara, ['i']))  # Laplace -> valore > 0
4
5 frase_rara = word_tokenize("i xyzword lol")
6 pp_mle = calcola_perplexity(modello_tweet, frase_rara, n) # MLE -> inf
7 pp_lap = calcola_perplexity(modello_tweet_lp, frase_rara, n)

```

Tabella 4: Confronto MLE vs. Laplace su frase con parola mai vista

Metrica	MLE	Laplace
$P(\text{xyzword} i)$	0.000000	0.000065
Perplexity su frase rara	∞	1111.48

Il MLE è inutilizzabile per calcolare la perplexity su frasi con parole fuori vocabolario (basta una singola parola ignota per rendere l'intera frase impossibile secondo il modello, con perplexity infinita), mentre Laplace garantisce sempre un valore finito.

10 Confronto $n = 1, 2, 3$

Tutti e tre i modelli sono addestrati solo sul corpus Twitter (`sequenze_tweet`), e la stessa frase viene valutata sia in-dominio (Twitter) sia fuori dominio (letteraria):

Confronto della perplexity al variare di n

```
1 frase_tw = word_tokenize("i love you so much thank you")
2 frase_lt = word_tokenize("she had been very well disposed")
3
4 for n_test in [1, 2, 3]:
5     tr_tmp, voc_tmp = padded_everygram_pipeline(n_test, sequence_tweet)
6     mod_tmp = Laplace(n_test)
7     mod_tmp.fit(tr_tmp, voc_tmp)
8
9     pp_tw = calcola_perplexity(mod_tmp, frase_tw, n_test)
10    pp_lt = calcola_perplexity(mod_tmp, frase_lt, n_test)
11    print(f"n={n_test}: PP Twitter={pp_tw:.2f}, PP Letterario={pp_lt:.2f}")
```

Tabella 5: Perplexity Laplace al variare di n (modelli addestrati solo su Twitter)

n	PP frase Twitter	PP frase Letteraria
1	124.04	1727.63
2	128.14	8059.42
3	307.20	10003.72

All'aumentare di n :

- La perplexity aumenta per *entrambe* le frasi, non solo per quella fuori dominio: con n maggiore lo smoothing di Laplace deve distribuire probabilità su uno spazio di n -grammi molto più grande ($|V|^n$), e la maggior parte non è mai stata osservata nel training. Questo effetto di **data sparsity** penalizza anche la frase in-dominio.
- L'aumento è però molto più marcato per la frase letteraria (fuori dominio): il divario tra le due perplexity si allarga passando da $n = 1$ a $n = 2$, segno che il modello sta imparando dipendenze sempre più specifiche del corpus Twitter, che la frase letteraria non rispetta.
- Questo mostra un compromesso pratico: aumentare n rende il modello più specifico al proprio dominio (utile per distinguere i due stili), ma anche più fragile per data sparsity quando applicato a un corpus di addestramento di dimensioni moderate.

11 Analisi Linguistica Comparata

11.1 Type-Token Ratio

Il TTR non è confrontabile calcolandolo sull'intero corpus di ciascun dominio: è una quantità che *scende* strutturalmente all'aumentare del numero di token (le parole nuove si esauriscono via via che il testo prosegue), quindi confrontare il TTR di $\sim 95\,000$ token di tweet con quello di $\sim 158\,000$ token di *Emma* misura soprattutto quale corpus è più piccolo, non quale sia lessicalmente più vario. Il TTR va quindi calcolato su campioni di uguale dimensione:

Calcolo del TTR su campioni di uguale dimensione

```
1 N = min(len(tokens_tweet), len(tokens_lett))
2 ttr_tweet = len(set(tokens_tweet[:N])) / N
3 ttr_lett = len(set(tokens_lett[:N])) / N
```

Su un campione di $N = 94706$ token (la dimensione dell'intero corpus Twitter, il più piccolo dei due) si ottiene $TTR_{\text{Twitter}} = 0.1270$ e $TTR_{\text{Letterario}} = 0.0719$. Twitter mantiene un TTR più alto anche a parità di token campionati (nonostante il divario si riduca rispetto al confronto non corretto, che dava 0.1270 vs. 0.0588): il vocabolario resta eterogeneo per la presenza di slang, hashtag, nomi propri e abbreviazioni, mentre il testo letterario, anche su un campione della stessa dimensione, usa un vocabolario relativamente più ristretto e ricorrente.

11.2 Lunghezza media delle frasi

Calcolo della lunghezza media delle frasi

```
1 lung_tw = [len(word_tokenize(t)) for t in tweets_raw]
2 frasi_lt = sent_tokenize(testo_letterario_raw)
3 lung_lt = [len(word_tokenize(f)) for f in frasi_lt]
4
5 media_tw = sum(lung_tw) / len(lung_tw)
6 media_lt = sum(lung_lt) / len(lung_lt)
```

Le frasi letterarie sono circa il doppio più lunghe dei tweet: il romanzo di Austen è ricco di subordinate e costruzioni relative, mentre i tweet tendono a essere telegrafici.

12 Risultati e Discussione

La Figura 1 raccoglie in un unico pannello i risultati chiave discussi nelle sezioni precedenti: i bigrammi più frequenti nei due domini (Sezione 4.2), il confronto diretto di log-likelihood sulle quattro frasi di test (Sezione 8) e le metriche linguistiche strutturali (TTR e lunghezza media, Sezione 11).

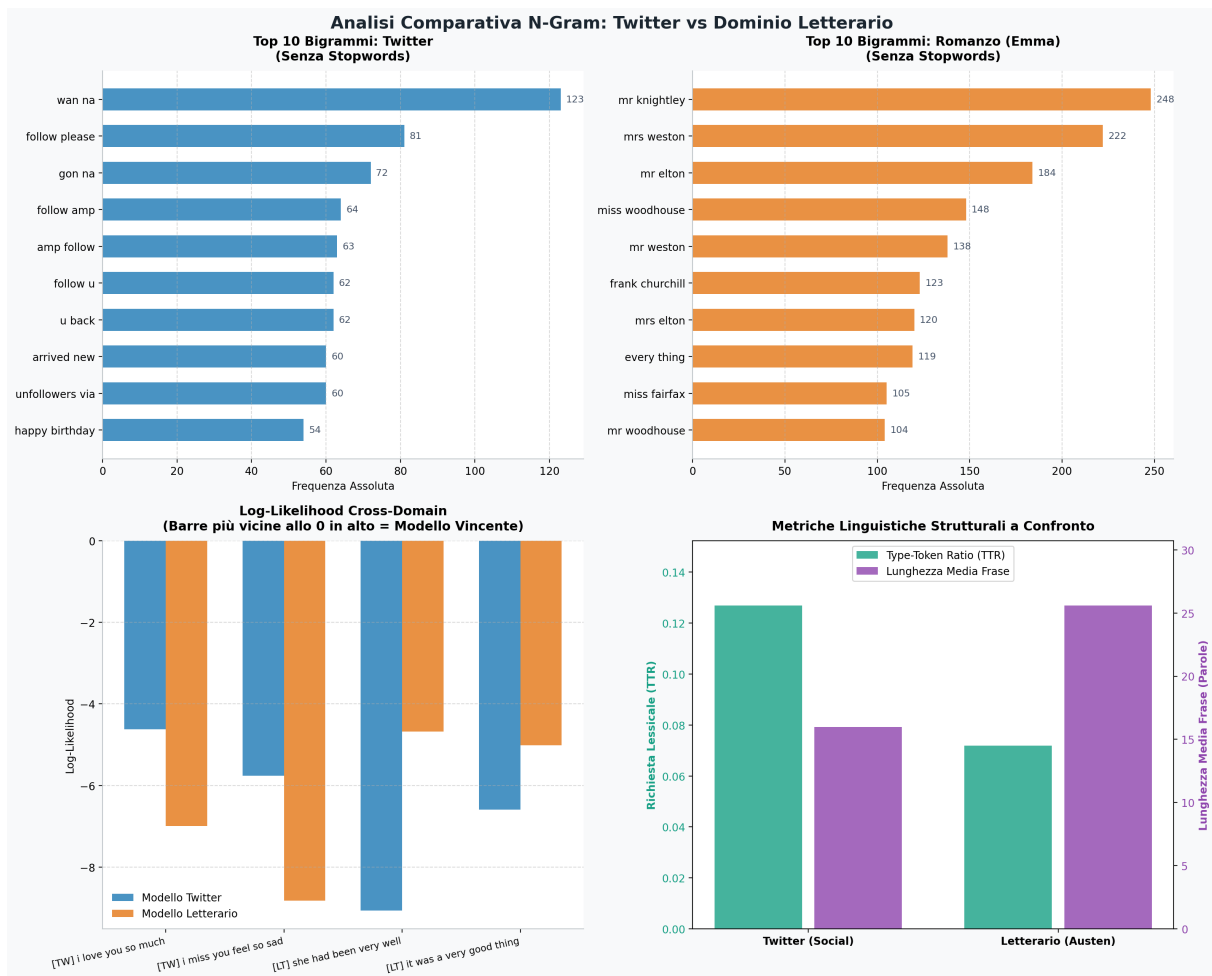


Figura 1: Riepilogo visivo: top-10 bigrammi per dominio (senza stopwords), log-likelihood cross-domain sulle quattro frasi di test, e confronto di TTR e lunghezza media frase.

12.1 Perché entrambi i classificatori raggiungono il 100%

Sia la perplexity normalizzata (Sezione 7) sia la log-likelihood (Sezione 8) classificano correttamente *tutte* le frasi di test. Il motivo è strutturale: il vocabolario di Twitter (slang, forme contratte, hashtag) e quello di Austen (registro ottocentesco, nomi propri come *Mr. Knightley*) si sovrappongono solo in un nucleo comune di parole funzionali, per cui una frase di un dominio contiene quasi sempre bigrammi mai osservati nel modello dell'altro — con Laplace questo si traduce in una perplexity nettamente peggiore per il modello “sbagliato”.

Questo spiega anche perché il vantaggio teorico della log-likelihood (non richiede la normalizzazione ad hoc per $|V|$, Sezione 7.2) non si traduca qui in un'accuratezza misurabilmente superiore: il compito è troppo facile per discriminare fra i due metodi. Un test più severo userebbe frasi di confine (es. testo giornalistico, a metà fra i due registri) invece di frasi nettamente appartenenti a un dominio.

12.2 Il ruolo dello smoothing: non solo evitare probabilità zero

Il confronto MLE vs. Laplace (Sezione 9) mostra che lo smoothing non è solo un accorgimento anti-crash, ma cambia concretamente l'output: su una parola mai vista, MLE

assegna probabilità 0 e perplexity ∞ (inutilizzabile per classificare o confrontare frasi), mentre Laplace assegna una probabilità piccola ma finita (6.5×10^{-5}) e una perplexity finita (1111.48). Senza smoothing, l'intera pipeline di valutazione delle Sezioni 7 e 10 sarebbe stata impossibile su dati reali, dove le parole fuori vocabolario sono la norma.

12.3 L'ordine n non aiuta linearmente: il costo della sparsity

Il confronto fra $n = 1, 2, 3$ (Sezione 10, Tabella 5) mostra che la perplexity *aumenta* con n per entrambe le frasi, non solo per quella fuori dominio: la PP Twitter passa da 124.04 a 307.20 ($n=1 \rightarrow 3$), perché lo spazio degli n -grammi cresce come $|V|^n$ mentre i dati di training restano fissi (data sparsity). L'effetto è però molto più marcato fuori dominio: la PP letteraria passa da 1727.63 a 10003.72 (fattore $\sim 5.8\times$ contro $\sim 2.5\times$ per Twitter). Aumentare n non rende quindi un modello “migliore” in assoluto, ma più specifico al proprio dominio — utile per un classificatore, ma rischioso per generazione o autocompletamento, dove il testo in input non sempre coincide con lo stile del training.

12.4 Sintesi tabellare

Tabella 6: Riepilogo delle caratteristiche dei due domini

Caratteristica	Twitter	Letterario
Dataset	10 000 tweet	<i>Emma</i> , Austen (1816)
TTR (campioni pari)	alto (0.1270)	basso (0.0719)
Lunghezza media frase	16.0 token	25.6 token
Bigrammi tipici	emozioni, auguri	titoli, azioni
Smoothing consigliato	Laplace	Laplace

12.5 Osservazioni principali

1. **I modelli N-gram riflettono lo stile del corpus:** il testo generato da ciascun modello è chiaramente riconoscibile come appartenente al dominio di addestramento.
2. **Entrambi i classificatori raggiungono il 100% di accuratezza** (Sezione 12.1): un risultato dovuto alla forte distanza lessicale fra i due domini più che a un vantaggio specifico di un metodo sull'altro.
3. **Laplace è indispensabile per la robustezza** (Sezione 12.2): MLE è ottimo per stimare probabilità su dati visti, ma non può gestire frasi di test con parole nuove.
4. **$n = 2$ è un buon compromesso:** cattura dipendenze locali senza soffrire troppo di sparsity, a differenza di $n = 3$.
5. **Twitter è linguisticamente più vario ma meno strutturato:** TTR più alto anche a parità di token campionati, frasi brevi, bigrammi emotivi. Austen ha TTR più basso ma struttura sintattica ricca.

13 Conclusioni

Il laboratorio ha dimostrato come i modelli linguistici basati su N-grammi siano in grado di catturare differenze stilistiche significative tra due domini molto distanti. Il modello addestrato su Twitter genera testo informale e ripetitivo, mentre quello addestrato su Austen produce frasi più formali e articolate.

Sia la perplexity normalizzata sia la log-likelihood classificano correttamente il 100% delle frasi di test: la distanza linguistica tra i due domini è tale da rendere il compito facile per entrambi i metodi. La log-likelihood resta preferibile sul piano metodologico perché confronta i due modelli senza bisogno di una normalizzazione ad hoc per la dimensione del vocabolario. Le tecniche di smoothing (in particolare Laplace) restano comunque essenziali per rendere i modelli applicabili a testi reali che contengono inevitabilmente parole fuori vocabolario.

14 Dichiarazione sui Contributi di IA

Per questo laboratorio ci siamo appoggiati anche a uno strumento di intelligenza artificiale generativa (Claude, Anthropic), usato come supporto puntuale durante lo sviluppo e non come autore del lavoro. Qualche esempio concreto di come lo abbiamo usato:

- **Snellimento dei commenti nel codice:** molti commenti erano troppo lunghi o ripetevano quanto già scritto nel testo; li abbiamo resi più diretti (es. perché serve addestrare un modello Laplace separato da MLE, o cosa misura esattamente la perplexity normalizzata per vocabolario).
- **Individuazione e correzione di due bug metodologici:** `perplexity()` di NLTK si aspetta in input n-grammi, non token grezzi (passandole i token grezzi calcola probabilità su singoli caratteri anziché su parole), e i modelli venivano addestrati su un'unica sequenza per l'intero corpus invece che una per ciascun tweet/frase, generando bigrammi spuri a cavallo tra un tweet e il successivo. Dopo la correzione abbiamo rilanciato lo script e aggiornato tutte le tabelle di perplexity con i valori reali (Sezioni 7, 9, 10).
- **Verifica di piccoli dettagli di correttezza:** rilettura del codice per controllare che pipeline di preprocessing, addestramento e valutazione fossero coerenti tra i due domini (Twitter e letterario).
- **Correzione di un'incoerenza tra codice e docstring:** il classificatore a log-likelihood veniva chiamato con i modelli MLE anziché Laplace, in contraddizione con la docstring della funzione; con MLE un bigramma mai visto attiva un floor artificiale ($\log 10^{-10}$) che di fatto decide la classificazione al posto della verosimiglianza reale. Abbiamo anche normalizzato lo score per la lunghezza della frase, per confrontare correttamente frasi di lunghezza diversa.
- **Correzione del confronto TTR:** il Type-Token Ratio calcolato sull'intero corpus non era confrontabile tra i due domini (dipende dalla dimensione del corpus); ricalcolato su campioni di uguale dimensione (Sezione 11.1).

L'impostazione del lavoro, le scelte metodologiche, l'interpretazione critica dei risultati e la stesura della relazione restano nostre.